MAESTRÍA EN MINERÍA DE DATOS · UTN FACULTAD REGIONAL ROSARIO

CLASE 6 · UNIDAD 3

Árboles de decisión

y Random Forest

Particiones recursivas, ensambles por bagging y la doble aleatorización: el paso del modelo lineal al bosque aleatorio en problemas financieros.

Dr. Darío Ezequiel Díaz

Jueves 14 de mayo de 2026 · 18:00 – 22:00 hs · Modalidad virtual

Plan de la sesión

Cuatro bloques articulados que recorren los fundamentos de los árboles CART, la matemática de la impureza, la construcción de ensambles por bagging y la implementación práctica del Random Forest en Python y R.

01

Árboles CART

Criterios de partición (Gini, entropía), poda preventiva y post-poda por cost-complexity. Interpretabilidad visual.

02

Sobreajuste y regularización

Diagnóstico del overfitting. Estrategias de control: max_depth, min_samples_leaf, ccp_alpha. Trade-off sesgo–varianza.

03

Random Forest

Bagging, doble aleatorización (bootstrap + subset de features), error OOB y agregación de votos.

04

Importancia y práctica

MDI versus permutation importance. Implementación en sklearn y rpart/randomForest. Comparativa con regresión logística.

¿Qué construimos la semana pasada?

Sigmoide y logit

$P(Y=1|X) = \sigma(\beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p)$. El logit conecta probabilidad con predictor lineal. Estimación por máxima verosimilitud.

Odds ratio

$\exp(\beta)$ traduce coeficientes a un lenguaje de negocio: cuánto cambian los odds del evento por un incremento unitario en la covariable.

Scorecard regulatoria

Construcción siguiendo Siddiqi: PDO=20, anclaje 600/50:1. Variables WoE, puntajes parciales por atributo. Estándar Basilea II/III.

AUC · KS · Gini

Las tres métricas mínimas que la industria exige de cualquier modelo de scoring crediticio. Hoy ampliamos el menú a árboles y bosques.

Hoy abandonamos el modelo lineal interpretable y entramos en la lógica de las particiones recursivas y los ensambles.

Posicionamiento en la Unidad 3

La Unidad 3 despliega el arsenal completo de algoritmos de clasificación supervisada en tres clases consecutivas. Hoy abordamos el tramo intermedio: árboles y bosques.

CLASE 5 · 07/05

Regresión logística

Scorecards · Basilea

CLASE 6 · HOY

CART · Random Forest

Particiones y bagging

CLASE 7 · 21/05

Boosting · SVM · MLP

Ensambles avanzados y redes

Hoy: los modelos basados en particiones que dominan los pipelines productivos de credit scoring en Argentina y el mundo.

Del modelo lineal al modelo basado en particiones

La regresión logística asume una frontera de decisión lineal en el espacio de los logits. Los árboles abandonan ese supuesto y construyen fronteras ortogonales que pueden capturar interacciones complejas sin especificarlas.

MODELO LINEAL

Regresión logística

- Frontera global lineal en logits.
- Coeficientes con interpretación económica directa (odds ratio).
- Requiere supuestos: linealidad, no multicolinealidad, independencia.
- Estándar regulatorio en banca.
- Limitado frente a interacciones no especificadas.

MODELO POR PARTICIONES

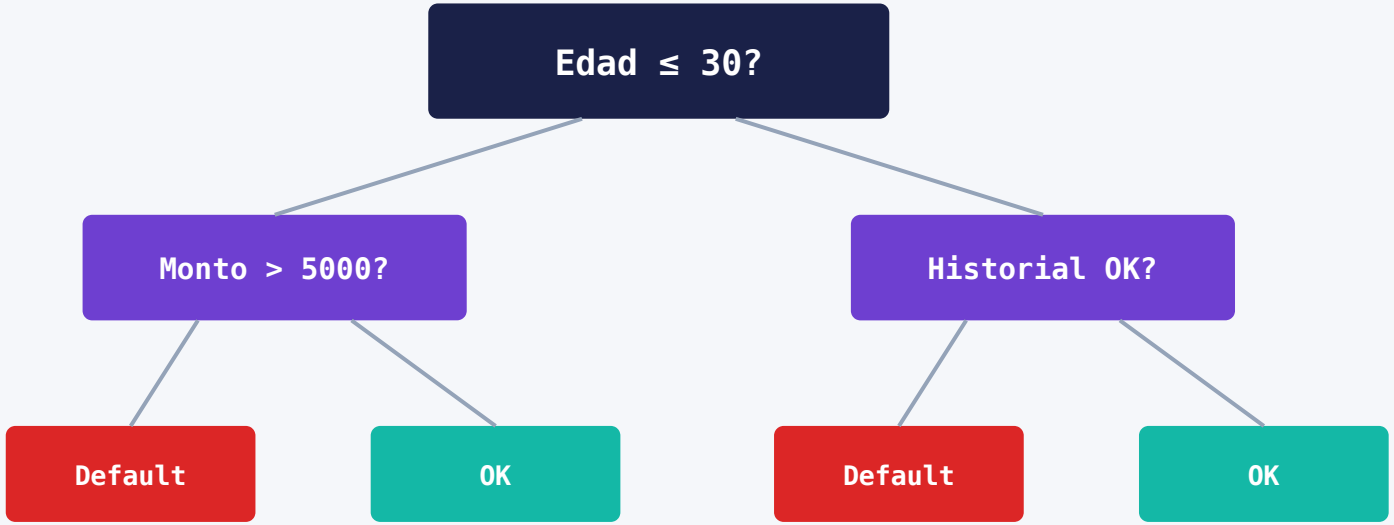
Árboles CART y ensambles

- Frontera por regiones rectangulares ortogonales.
- Capturan interacciones no lineales sin especificarlas.
- Libres de supuestos distribucionales.
- Manejan naturalmente variables categóricas y faltantes.
- Riesgo de sobreajuste — exigen poda o ensamblar.

Ambas familias son complementarias: la logística como benchmark interpretable; los árboles como herramienta exploratoria y los bosques como motor predictivo.

¿Qué es un árbol de decisión?

Un árbol de decisión particiona recursivamente el espacio de las covariables mediante reglas binarias simples. La estructura jerárquica resultante es fácilmente legible: cada camino desde la raíz hasta una hoja describe una regla de clasificación explícita.



LECTURA OPERATIVA

Cada camino es una regla

- Si edad ≤ 30 y monto > 5000 → Default
- Si edad ≤ 30 y monto ≤ 5000 → OK
- Si edad > 30 e historial malo → Default
- Si edad > 30 e historial bueno → OK

Anatomía de un árbol CART

Breiman, Friedman, Olshen y Stone (1984) formalizaron el algoritmo CART — Classification and Regression Trees — como un esquema binario y recursivo. Cuatro elementos estructurales conviene distinguir con claridad.

RAÍZ

Nodo raíz

Contiene la totalidad del conjunto de entrenamiento. El algoritmo evalúa todas las variables y todos los puntos de corte candidatos.

INTERNO

Nodo interno

Cada nodo aplica una regla binaria $X_j \leq c$. Las observaciones que la satisfacen pasan al hijo izquierdo; las restantes, al derecho.

TERMINAL

Nodo hoja

Las hojas contienen la predicción final: la clase mayoritaria (clasificación) o la media (regresión). No se subdividen más.

PROFUNDIDAD

Profundidad

Número de aristas entre la raíz y la hoja más lejana. Hiperparámetro crítico: controla la complejidad del modelo.

En cada nodo, CART selecciona el par (variable, punto de corte) que MAXIMIZA la reducción de impureza: $\Delta I = I(\text{padre}) - [p_L \cdot I(L) + p_R \cdot I(R)]$

Gini, entropía y la medida de impureza

La impureza de un nodo cuantifica la heterogeneidad de las clases en él contenidas. Un nodo es puro cuando todas las observaciones pertenecen a la misma clase. CART admite dos formulaciones equivalentes en la práctica.

$$\text{Gini}(t) = 1 - \Sigma\, p^2(i \mid t)$$

$$\text{Entropía}(t) = -\,\Sigma\, p(i \mid t) \cdot \log_2 p(i \mid t)$$

Mínimo = 0 (nodo puro) · Máximo cuando las clases están equidistribuidas en el nodo

GINI

Impureza de Gini

Probabilidad de clasificar incorrectamente una observación elegida al azar dentro del nodo, si la asignáramos según la distribución empírica. Rango [0, 0.5] en binario. Default en scikit-learn.

ENTROPÍA

Entropía de Shannon

Cantidad esperada de información necesaria para identificar la clase de una observación. Rango [0, $\log_2(K)$]. Su gradiente coincide con la pérdida log-loss.

Ejemplo numérico: ¿cómo se elige el mejor split?

Supongamos un nodo padre con 100 observaciones del German Credit: 70 buenos pagadores y 30 morosos. Evaluamos dos splits candidatos y comparamos su ganancia de información en términos de Gini.

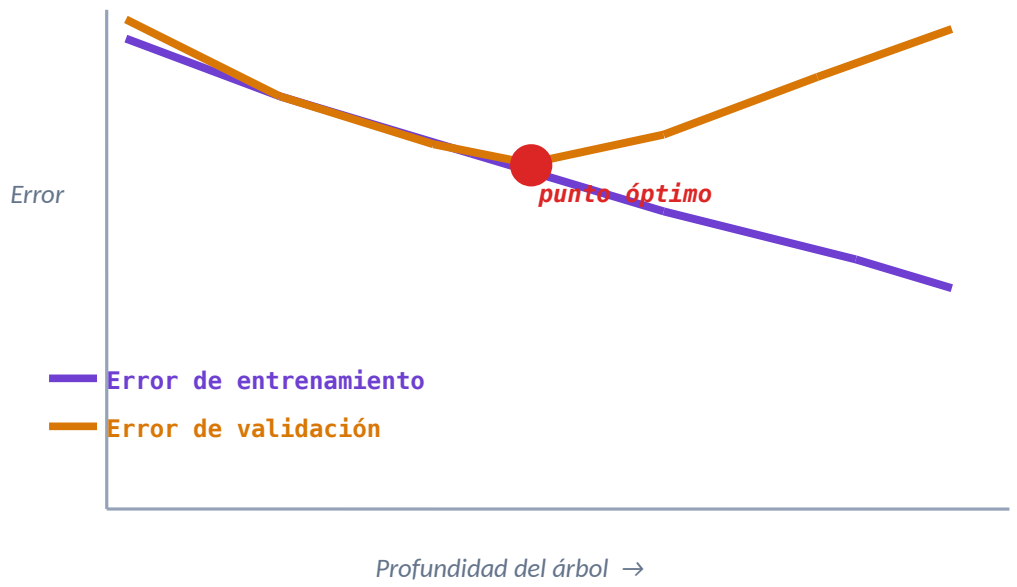
Candidato	Hijo izquierdo (L)	Hijo derecho (R)	Gini ponderado	Ganancia ΔI
Duración ≤ 12 m	n=40 · 35 bueno · 5 malo Gini = 0.219	n=60 · 35 bueno · 25 malo Gini = 0.486	0.379	+0.041
Historial = bueno	n=55 · 50 bueno · 5 malo Gini = 0.165	n=45 · 20 bueno · 25 malo Gini = 0.494	0.313	+0.107

Nodo padre: n = 100 · 70 buenos · 30 morosos · Gini(padre) = 0.420

CART selecciona el split sobre Historial: produce mayor reducción de impureza (ΔI = +0.107). Repite recursivamente en cada nodo hijo hasta el criterio de parada.

El sobreajuste: enemigo número uno

Un árbol sin restricciones crece hasta lograr nodos puros — en el límite, una hoja por observación. La precisión de entrenamiento alcanza el 100 %, pero la generalización colapsa. Esta es la patología característica de los árboles individuales.



DIAGNÓSTICO

Síntomas del overfitting

- Accuracy en train cercana a 1.0, en test mucho menor.
- Hojas con muy pocas observaciones (1 o 2).
- Profundidad excesiva (más de 8–10 niveles).
- Inestabilidad: pequeños cambios en los datos producen árboles muy distintos.
- Sensibilidad extrema a outliers.

Pre-poda y post-poda

Existen dos filosofías para controlar el crecimiento de un árbol. La pre-poda detiene la expansión durante el entrenamiento; la post-poda permite que el árbol crezca y luego elimina ramas que no aportan capacidad predictiva.

PRE-PODA · EARLY STOPPING

Detener antes de crecer

Se imponen restricciones que detienen la expansión cuando ya no aporta una mejora suficiente.

- max_depth — profundidad máxima permitida.
- min_samples_split — observaciones mínimas para dividir.
- min_samples_leaf — observaciones mínimas por hoja.
- min_impurity_decrease — mejora mínima de Gini.

POST-PODA · COST-COMPLEXITY

Crecer y luego podar

Se permite el crecimiento completo; luego se eliminan ramas penalizando complejidad mediante un parámetro α.

$$R_\alpha(T) = R(T) + \alpha \cdot |\tilde{T}|$$

R(T): error sobre los datos · $|\tilde{T}|$: número de hojas · α: penalización por complejidad. Mayor α implica un árbol más pequeño.

Hiperparámetros que conviene ajustar

Pocos hiperparámetros gobiernan el comportamiento de un árbol de decisión. Los siguientes cinco concentran prácticamente todas las decisiones críticas para controlar el sobreajuste y la interpretabilidad.

Hiperparámetro	Función	Rango razonable	Efecto sobre el árbol
`max_depth`	Profundidad máxima permitida	3 – 10	Hiperparámetro más eficaz para controlar la complejidad global.
`min_samples_split`	Obs. mínimas para dividir un nodo	10 – 50	Evita splits sobre muestras pequeñas y reduce ruido.
`min_samples_leaf`	Obs. mínimas en cada hoja terminal	5 – 30	Garantiza hojas estadísticamente significativas.
`ccp_alpha`	Penalización por complejidad	0.0 – 0.05	Post-poda cost-complexity. Mayor $\alpha \rightarrow$ árbol más compacto.
`criterion`	Función de impureza	gini / entropy	Diferencias prácticas mínimas; Gini algo más rápido.

Recomendación práctica: tunear primero max_depth y min_samples_leaf por grid search; ccp_alpha en una segunda pasada de refinamiento.

Cuándo preferir CART (y cuándo desconfiar)

El árbol de decisión es la herramienta exploratoria por excelencia: rápida, visual, comunicable. Pero su uso aislado en producción exige cuidados — la inestabilidad es su talón de Aquiles.

FORTALEZAS

Lo que CART hace bien

- Interpretabilidad visual inmediata — cada regla es explicable.
- Capta interacciones no lineales sin especificarlas.
- Tolera variables categóricas, faltantes y outliers.
- No requiere supuestos distribucionales ni escalado.
- Selección implícita de variables relevantes.
- Predicción extremadamente rápida en producción.

LIMITACIONES

Cuándo desconfiar

- Alta varianza: pequeños cambios en los datos producen árboles distintos.
- Sesgo hacia variables con muchas categorías o cortes.
- Fronteras ortogonales — pobres con relaciones diagonales.
- Tendencia natural al sobreajuste sin poda agresiva.
- Optimización greedy — no garantiza el árbol óptimo global.
- Performance predictiva inferior a ensambles modernos.

De un árbol a un bosque: la sabiduría del ensamble

Si un solo árbol exhibe alta varianza, ¿qué ocurre cuando agregamos las predicciones de muchos árboles independientes entrenados sobre versiones perturbadas del dataset? Galton lo observó en 1907 con el peso de un buey; Breiman lo formalizó en 2001 para el aprendizaje estadístico.

INSIGHT 1

Reducción de varianza

Si tenemos B árboles iid con varianza σ^2, el promedio tiene varianza σ^2/B. La predicción agregada es más estable que cualquier árbol individual.

INSIGHT 2

Bootstrap aggregating

Cada árbol se entrena sobre una muestra bootstrap del set de entrenamiento (con reemplazo, n observaciones). Esto induce diversidad entre los árboles.

INSIGHT 3

Decorrelación

En cada split, sólo se evalúa un subconjunto aleatorio de variables. Esta segunda aleatorización rompe la correlación entre árboles.

$$\hat{y}_{RF}(x) = \text{mode}(\hat{y}_1(x), \hat{y}_2(x), \ldots, \hat{y}_B(x)) \quad \rightarrow \quad \text{voto mayoritario sobre B árboles diversificados}$$

Random Forest: la doble aleatorización

Leo Breiman (2001) propuso introducir aleatoriedad en dos niveles simultáneamente. Esta doble inyección de varianza paradójicamente reduce la varianza del ensamble final mediante la decorrelación de los árboles individuales.

ALEATORIZACIÓN 1 · BOOTSTRAP

Muestras con reemplazo

Cada árbol $b \in \{1, \ldots, B\}$ se entrena sobre una muestra bootstrap D_b extraída con reemplazo del set de entrenamiento.

- n observaciones muestreadas con reemplazo de D.
- Aproximadamente el 63.2 % de obs. únicas por muestra.
- El 36.8 % restante: muestra out-of-bag (OOB).
- Cada D_b produce un árbol T_β distinto.

ALEATORIZACIÓN 2 · FEATURES

Subconjunto de variables

En cada partición de cada árbol, sólo se evalúa un subconjunto $m \ll p$ de variables aleatoriamente seleccionado.

- $m = \sqrt{p}$ para clasificación (default sklearn).
- $m = p/3$ para regresión.
- Si una variable es muy fuerte, no domina todos los árboles.
- Decorrelación → reducción efectiva de la varianza.

Hiperparámetros del Random Forest

Random Forest hereda los hiperparámetros de CART y suma los específicos del ensamble. El tunning suele ser más amable que en gradient boosting: el modelo es relativamente robusto frente a la elección de los parámetros.

Hiperparámetro	Default sklearn	Rango razonable	Lectura
`n_estimators`	100	100 – 500	Más árboles → menor varianza, pero costo lineal. Diminishing returns más allá de 300.
`max_features`	sqrt(p)	sqrt, log2, 0.3–0.5	Controla decorrelación. Menor m → árboles más diversos.
`max_depth`	None	None, 10–30	Por default crecen sin límite — el bagging controla la varianza.
`min_samples_leaf`	1	1 – 10	Aumentarlo aumenta sesgo y reduce ruido en problemas pequeños.
`bootstrap`	True	True	Mantener en True. Habilita la estimación OOB del error.
`class_weight`	None	'balanced'	Crítico en problemas desbalanceados (fraude, default).

Importancia de variables: MDI vs. permutation

Un Random Forest pierde la interpretabilidad visual de un árbol único, pero ofrece a cambio una medida cuantitativa global de la relevancia de cada variable. Dos enfoques alternativos conviene conocer en detalle.

MDI · MEAN DECREASE IMPURITY

Importancia interna

Suma de las reducciones de impureza atribuibles a cada variable, ponderadas por el número de obs. en cada nodo, promediadas sobre todos los árboles.

- Cómputo gratuito durante el entrenamiento.
- Sesgada hacia variables con alta cardinalidad.
- Sesgada en presencia de variables correlacionadas.
- Atributo: model.feature_importances_

PERMUTATION IMPORTANCE

Importancia externa

Caída de la métrica de validación cuando los valores de una variable se permutan aleatoriamente, manteniendo fijo el modelo entrenado.

- Más costosa: requiere predecir tras cada permutación.
- Insesgada frente a la cardinalidad de la variable.
- Sigue siendo sensible a correlaciones (proxies).
- Función: sklearn.inspection.permutation_importance

Error OOB: validación cruzada gratuita

El muestreo bootstrap deja fuera, en promedio, el 36.8 % de las observaciones por árbol. Estas observaciones out-of-bag actúan como un conjunto de validación intrínseco — sin necesidad de reservar un fold adicional.

LÓGICA DEL CÁLCULO

Cómo se obtiene el error OOB

1. Para cada observación i, identificar los árboles que NO la usaron en su muestra bootstrap (≈ 37 % del bosque).
2. Predecir y_i agregando sólo esos árboles.
3. Calcular el error promediando sobre todas las observaciones.
4. El resultado aproxima un k-fold con k = B.

VENTAJAS

Sin reservar set de validación

El error OOB elimina la necesidad de validation split en datasets pequeños. Equivalente asintóticamente a un k-fold con k = B.

USO PRÁCTICO

Activación en sklearn

Setear oob_score=True al instanciar RandomForestClassifier. El error queda disponible en el atributo .oob_score_.

Comparativa sobre el German Credit

Ilustramos el comportamiento empírico esperable de los tres algoritmos sobre el mismo dataset con división 80/20. Las cifras son indicativas — la magnitud exacta depende del tunning y del split aleatorio.

Criterio	Reg. Logística	Árbol CART	Random Forest
AUC-ROC (test)	**0.78**	**0.72**	**0.82**
KS estadístico	0.41	0.35	0.48
Gini	0.56	0.44	0.64
Interpretabilidad	Alta (OR)	Muy alta (visual)	Media (SHAP)
Robustez frente a outliers	Baja	Alta	Muy alta
Tiempo de entrenamiento	Bajo	Muy bajo	Medio
Aceptación regulatoria	Estándar	Aceptable	Con SHAP

Cifras orientativas. RF supera a la logística en performance, pero la logística mantiene la ventaja en explicabilidad regulatoria y simplicidad operativa.

Implementación en Python (scikit-learn)

Dos modelos en pocas líneas. Notar el uso explícito de random_state=2026 para reproducibilidad y oob_score=True en el Random Forest para obtener gratis la estimación interna del error.

Python · CART (DecisionTreeClassifier)

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

cart = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_leaf=20,
    ccp_alpha=0.005,
    random_state=2026)

cart.fit(X_train, y_train)
p_hat = cart.predict_proba(X_test)[:, 1]

# Visualizar el árbol
plot_tree(cart, feature_names=cols,
          class_names=['OK','Default'])
```

Python · Random Forest (RandomForestClassifier)

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance

rf = RandomForestClassifier(
    n_estimators=300,
    max_features='sqrt',
    min_samples_leaf=5,
    class_weight='balanced',
    oob_score=True,
    n_jobs=-1, random_state=2026)

rf.fit(X_train, y_train)
print(f'OOB score = {rf.oob_score_:.4f}')

# Importancia por permutación
perm = permutation_importance(rf, X_test, y_test,
                              n_repeats=30,
random_state=2026)
```

Implementación en R (rpart + randomForest)

Los paquetes rpart (Therneau y Atkinson) y randomForest (Breiman, Cutler y Liaw) constituyen el referente histórico en R. Como alternativa moderna existe el ecosistema tidymodels.

R · rpart + rpart.plot

```r
library(rpart)
library(rpart.plot)

set.seed(2026)

cart <- rpart(Y ~ ., data = train,
  method  = 'class',
  parms   = list(split = 'gini'),
  control = rpart.control(
    maxdepth = 5, minbucket = 20, cp = 0.005))

# Visualización del árbol
rpart.plot(cart, type = 4, extra = 104,
           box.palette = 'GnRd')

# Predicción de probabilidades
p_hat <- predict(cart, test, type = 'prob')[, 2]
```

R · randomForest

```r
library(randomForest)

set.seed(2026)

rf <- randomForest(
  Y ~ ., data = train,
  ntree     = 300,
  mtry      = floor(sqrt(ncol(train) - 1)),
  nodesize  = 5,
  classwt   = c('0' = 1, '1' = 3),
  importance = TRUE)

# Error OOB y diagnóstico
print(rf)
p_hat <- predict(rf, test, type = 'prob')[, 2]

# Importancia de variables
varImpPlot(rf, sort = TRUE, n.var = 10)
```

Términos clave de la Clase 6

Doce términos que deben integrarse al vocabulario profesional después de esta sesión. Su dominio resulta indispensable para abordar la Clase 7 (boosting, SVM, redes) y el proyecto integrador.

CART	Classification and Regression Trees (Breiman 1984)
Entropía	Medida de impureza basada en log-probabilidades
ccp_alpha	Penalización por complejidad en cost-complexity pruning
Bootstrap	Muestreo con reemplazo de tamaño n
MDI	Mean Decrease in Impurity — importancia interna
mtry	Número de variables sorteadas por split ($m \ll p$)
Gini	Medida de impureza basada en probabilidades cuadráticas
Poda	Reducción de complejidad del árbol entrenado
Bagging	Bootstrap aggregating (Breiman 1996)
OOB	Out-of-bag: ≈ 36.8 % de obs. no muestreadas
Permutation Importance	Caída de la métrica al permutar la variable
Voto mayoritario	Agregación de B clasificadores en RF

CIERRE DE LA CLASE 6

Actividad práctica y anticipación

ENTREGA PARCIAL · CLASE 7

Caso: árboles y bosques sobre German Credit

- Entrenar un CART con max_depth=5 y poda por ccp_alpha.
- Entrenar un Random Forest con n_estimators=300, max_features='sqrt'.
- Comparar AUC, KS y Gini con la logística de la Clase 5.
- Reportar las 10 variables más importantes (MDI y permutation).
- Interpretar económicamente las 3 variables top.

Formato: Notebook en Colab (Python o R). Entrega: Clase 7.

PRÓXIMA CLASE — JUE 21/05

Clase 7: Boosting, SVM y Redes Neuronales

- Gradient Boosting: aprendizaje secuencial sobre residuos.
- XGBoost: regularización, sparsity-aware split, tree pruning.
- SVM: kernel trick, hiperplano de margen máximo.
- Redes neuronales: MLP, funciones de activación, backpropagation.
- Calibración con grid search y random search.

Del bagging al boosting: la otra gran familia de ensambles.

LECTURA SUGERIDA: Apunte de Cátedra · Unidad 3 · secciones 3.2 y 3.3. *Breiman, L. (2001) Random Forests. Hastie, Tibshirani, Friedman (2009) ESL, cap. 9 y 15.*